INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA
SALT LAKE
(Affiliated to UEM Kolkata and Approved by AICTE)

Advanced Programming Lab

[PCCCS495]

Library Management System

Name: -Dipam Bhattacharya

Semester: -IV

Class Roll: - 57

Enrolment Number: - 12024052004054

Date: - 29/03/2026

Instructor name: - Dr.Susovan Jana, DR.Soumadip Biswas, Dr.Swagatam Basu

Department of Information Technology

2026

2. Problem Statement:

Managing library records manually is time-consuming and prone to errors. Traditional methods lack efficiency in tracking book availability, issuing records, and returning processes.

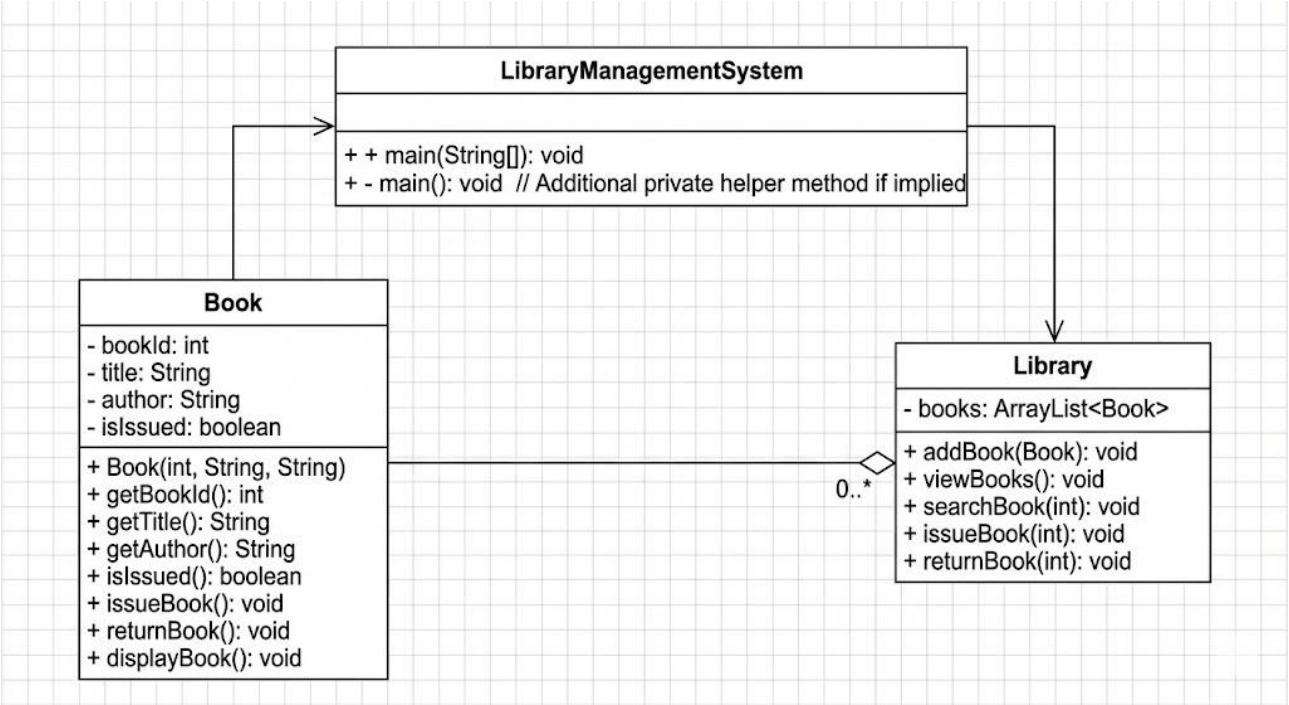
This project aims to develop a console-based Library Management System using Java. The system allows users to:

- Add new books
- View all books
- Search for books
- Issue books
- Return books

The goal is to create a structured, scalable, and efficient system using Object-Oriented Programming (OOP) principles.

3. System Design:

3.1 UML Class Diagram (Textual Representation):



3.2 Description of Major Classes:

i. Book Class

- Represents a book entity.
- Contains attributes: bookId, title, author, and issue status.
- Provides methods to issue, return, and display book details.

ii. Library Class

- Manages a collection of books.
- Handles operations like adding, searching, issuing, and returning books.

iii. Main Class

- Entry point of the application.
- Provides menu-driven interaction with the user.

3.4 Interaction Flow:

1. User starts the program
2. Menu is displayed
3. User selects an option:
    - Add Book → stored in list
    - View Books → display all books
    - Search Book → find by ID
    - Issue Book → update status
    - Return Book → update status
4. System continues until user exits

---

4. OOP Concepts Demonstrated:

4.1 Encapsulation

Where used: Book class
Why used: To protect data and restrict direct access

Code Snippet:

```
private int bookId;


public int getBookId() {
   return bookId;
}
```

Explanation:
Data members are declared private and accessed using getter methods, ensuring controlled access.

4.2 Abstraction

Where used: Library methods
Why used: To hide internal implementation details

Code Snippet:

```
public void issueBook(int id) {
    for (Book book : books) {
        if (book.getBookId() == id) {
            book.issueBook();
        }
    }
}
```

Explanation:
User interacts with simple methods while internal logic

4.3 Association (HAS-A Relationship)

Where used: Library contains Book objects
Why used: To represent real-world relationship

Code Snippet:

```
private ArrayList<Book> books = new ArrayList<>();
```

Explanation:
Library stores multiple Book objects, forming a HAS-A relationship.

4.4 Object Creation

Where used: Main class

Code Snippet:

```
Library library = new Library();

library.addBook(new Book(1, "Java Programming", "James Gosling"));
```

Explanation:
Objects are created dynamically to represent books and library.

---

5. Implementation Highlights:

5.1 Adding Books

```
public void addBook(Book book) {
    books.add(book);
}
```

Justification:
Allows dynamic addition of books into the system.

5.2 Searching Books

```
public void searchBook(int id) {
    for (Book book : books) {
        if (book.getBookId() == id) {
            book.displayBook();
        }
    }
}
```

Justification:
Efficient for small datasets and simple implementation.

5.3 Issue Book Logic

```
if (!book.isIssued()) {
    book.issueBook();
}
```

Justification:
Prevents issuing an already issued book.

5.4 Return Book Logic

```
if (book.isIssued()) {
    book.returnBook();
}
```

Justification:
Ensures valid return operations.

---

6. Testing & Error Handling:

6.1 Test Cases

Test Case	Input	Expected Output
Add Book	Valid data	Book added
Search Book	Valid ID	Book found
Search Book	Invalid ID	Not found
Issue Book	Available	Issued
Issue Book	Already issued	Error message

6.2 Edge Cases Handled

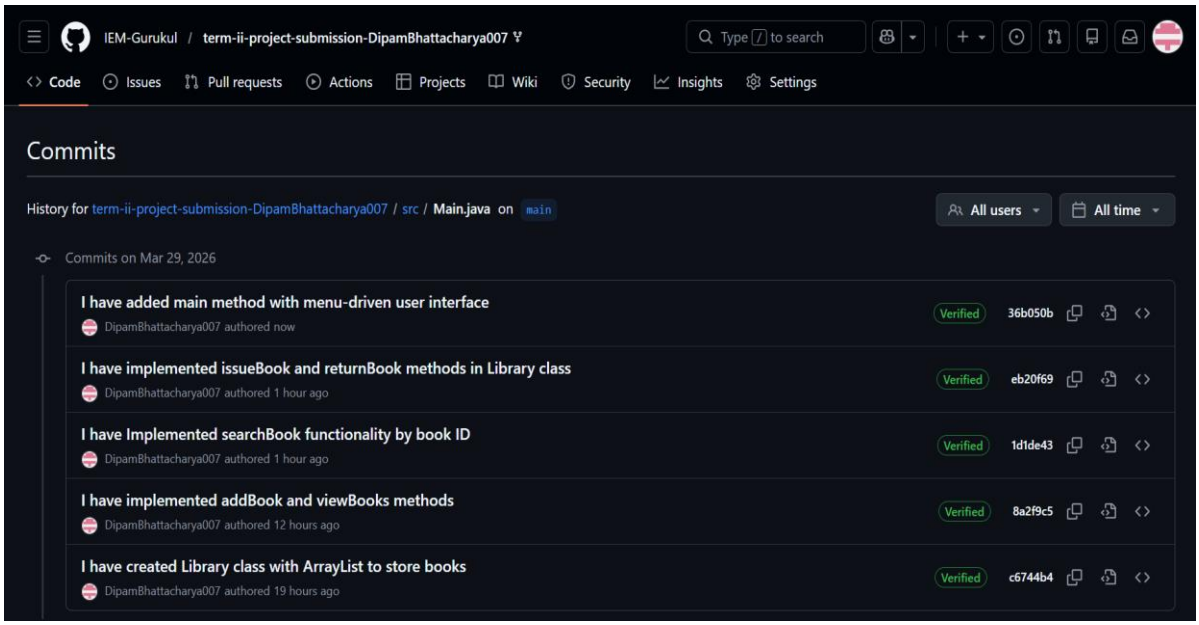
- No books in library
- Duplicate book IDs
- Invalid user input
- Empty data fields

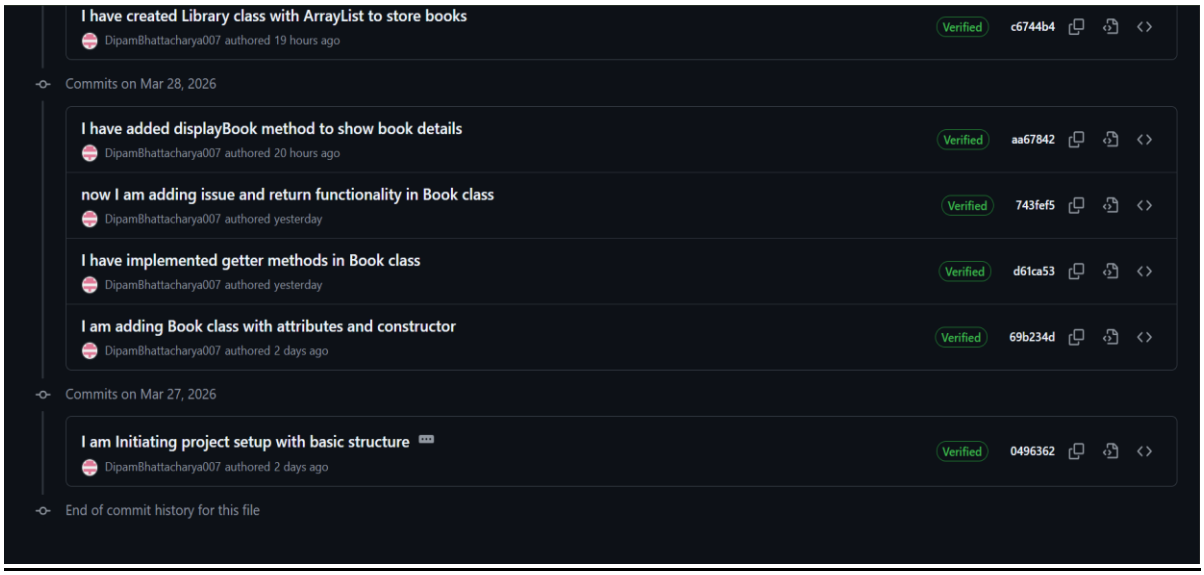
6.3 Failure Scenarios

- Book not found → handled with message
- Invalid menu choice → handled with default case
- Already issued book → prevented
- Returning non-issued book → prevented

---

7. Git Workflow:





---

8. Conclusion & Future Scope:

The Library Management System successfully demonstrates the use of Object-Oriented Programming concepts in building a structured and efficient application. It simplifies book management and improves usability through a console interface.

Future Improvements:

- GUI using Java Swing/JavaFX
- Database integration (MySQL)
- User login system
- Book reservation feature